

# TIN DANG HUYNH TRUNG

## Senior Software Engineer

@ tindang.ht97@gmail.com

+84 937 684 229

Ho Chi Minh City, Viet Nam

TinDang97

TinDang97

*I'm a low-key developer: a cup of coffee, a hard problem, and something useful at the end of it.*

Senior software engineer, ten years in — mostly Python, plus Rust and TypeScript. I've had machine learning in production since 2019: camera analytics and a vector search engine I wrote myself, then fintech pricing and language models, and for the last two years the infrastructure under LLM products — gateways, routing and failover, retrieval, agent memory, and metering that reconciles against what providers actually bill. I've been CTO of an AI startup and led teams of up to fourteen, and I still keep my hands on the hard parts. I'd rather measure than claim, so retrieval quality is gated in CI and inference is tuned against a P95. An agent writes most of my code now; deciding whether it's correct is still my job, and I published the method I use for that.

## PROFESSIONAL EXPERIENCE

### Technical Architect **Trustify Technology**

Jan 2024 – Present • Ho Chi Minh City

I architect and lead client platforms across Vietnam and Myanmar — transport, fintech, real estate and investment data for Yoma Group, Yoma Bank, Wave Money, Trustonic and VinaCapital — and wrote the company's engineering career ladder.

**Real-time ML serving at the edge** • *intelligent transport — architect & sole engineer, Sep 2025 – Mar 2026*

- **85 ms P95 inference across 400 camera streams.** YOLO runs on TensorRT in half precision behind Triton with dynamic batching, and falls back to ONNX Runtime where no GPU is present.
- **75% less backhaul**, from about 2.4 Gbps to 600 Mbps: inference and H.265 transcoding run at the roadside and only events travel on, over mutually authenticated Kafka into PyFlink, load-tested to 71K events a second.

Python • FastAPI • YOLO • TensorRT • Triton • ONNX Runtime • Kafka • PyFlink • K3s • ArgoCD • DuckDB

**Estate-finance platform & PCI boundary** • *Yoma Group, Wave Money — architect & sole engineer, Jul – Sep 2026*

- **One append-only double-entry ledger** replaced a live PHP/Laravel system whose balances nobody could audit: thirty modules in eight releases over four weeks, with 235 legacy business rules graded against it before any customer moved.
- **PCI scope cut to one service.** Cardholder data sits behind its own service, database and network, and the build rejects any import that crosses the boundary.

Python 3.12 • FastAPI • async SQLAlchemy 2 • PostgreSQL (schema per tenant) • Redis • 2C2P • AWS ECS • Envoy

**Also delivered:** a governed Databricks lakehouse for VinaCapital that took reporting from two days to two hours; the rollback path of Trustonic's per-customer IoT migration, where rehearsing it showed the document store, cache and archived objects did not revert with Postgres; a 2C2P card-payment integration for Wave Money; and Yoma Bank's microfinance lending MVP in Java 21 and Spring Boot.

### CTO – Co-founder **ClickAI Viet Nam**

Jun 2024 – Oct 2025 (concurrent) • Ho Chi Minh City

I co-founded ClickAI (clickai.vn) and built its conversational AI platform from scratch — multi-modal chat, image and audio generation, community — to 40K+ users across Southeast Asia inside four months, leading twelve to fourteen people and owning routing, billing and deployment myself.

- **20% lower API spend.** An orchestration layer routes each request across 20+ model providers by policy — task complexity, compliance, cost against quality — with the policies tested against each other in A/B runs.
- **Failover before users notice.** After a provider went hard-down and users felt it before routing shifted, I probed every provider continuously and marked one down before traffic could reach it.
- **Our own models behind the same gateway:** open-weight LLMs served with vLLM on rented GPUs, LangChain for prompt and agent orchestration, and image and audio generation behind circuit breakers.
- **80% faster retrieval** from hybrid keyword-and-vector search over Weaviate and Qdrant with citation tracking; a Graphiti knowledge graph on FalkorDB carries context from one conversation to the next.
- **Caught a 30% billing drift no alert had seen.** A retry loop was re-sending work; my reconciliation check against the providers' bills found it, and I keyed idempotency on the source event so a retry cannot double-count.
- **99.9% uptime, sub-2s responses, 1K+ concurrent sessions** on the 13-service Kubernetes deployment I authored; a three-node Patroni cluster made losing a database node a non-event, and a hosting comparison halved the bill.

Python • FastAPI • LangChain • vLLM • Weaviate • Qdrant • FalkorDB • PostgreSQL (partitioned, Patroni) • Redis • Kubernetes • Helm • Azure • Next.js

### Founder & Principal Engineer **Pilot Space (open-source AI infrastructure)**

Jan 2026 – Present • Remote

My own open-source line, each project published with its benchmark: **Lunaris** (agent memory; recall over 100K documents in 19–22 ms), **laya-codex** (code retrieval with a re-ranker I fine-tuned; right files in an agent's first answer up from 73% to 88%), **Moon** (Redis-compatible store with vector, full-text and graph search; 1.62× Redis on GET), **Hydroa** (multi-tenant LLM gateway) and **ADD** (my method for trusting agent-written code). Details and links are on the last page.

### Senior Data Engineer **Asia Commercial Bank (ACB)**

Aug 2023 – Jun 2024 • Ho Chi Minh City

- **80% less manual verification** on a data lake carrying 5M+ transactions a day, from a data-quality framework I built on Spark that checks each data asset across six dimensions; my FastAPI services fed **100+ internal applications**.

Cloudera • PySpark • Dagster • Iceberg • FastAPI

**Pricing, language & vision models** • *asset appraisal and lending platform*

An appraiser needed fifteen minutes to price a second-hand item by hand, from unstructured Vietnamese listings spread over more than fifty marketplaces.

- **Appraisal from 15 minutes to 5 seconds.** Price forecasting at 96% accuracy, served as an API on a platform carrying 10M+ daily transactions at 99.9% uptime.
- **96% F1 on Vietnamese entity extraction.** A named-entity model fine-tuned from BERT turns a listing into structured attributes; anomaly detection removes bad listings, and a 20K-node asset graph links products to their listings.
- **Crawler throughput doubled** to 1,000+ pages an hour across 50+ marketplaces on a distributed crawler I designed; I also started the company's open-source face-anti-spoofing.

Python · PyTorch · BERT · scikit-learn · Prophet · FAISS · Scrapy · Pub/Sub · BigQuery · Airflow · Kubernetes · GCP

**Software Developer (AI)** Dai Phat Solutions

Aug 2019 – Mar 2021 • Ho Chi Minh City

**Camera AI – vector search engine & distributed inference** • *architect of the camera-AI product*

Recognising faces across many cameras needs a similarity lookup on every frame, and the first pipeline showed video three to five seconds late – too slow for a security alert.

- **Delay cut from 3–5 s to 1 s** across 16 cameras at 15 FPS each: I tuned FFmpeg decoding, moved JPEG coding to TurboJPEG, compressed arrays on the wire with Blosc and reused pre-allocated memory for vectors.
- **Wrote the vector search engine** on FAISS: named indexes, flat and IVF with training, half-precision GPU indexes that move between GPU and CPU, persistence, and duplicate filtering on insert – reached through a Kafka request-and-reply client, with vectors held in Redis.
- **Optimised model inference.** I exported the detection and face-recognition models from PyTorch to TorchScript, ONNX Runtime and TensorRT, ran them in half precision, and batched frames from several cameras onto one GPU.
- **Designed it as a distributed system.** Decoding, detection, embedding and search workers pass frames over Kafka, Pulsar and ZeroMQ, so capacity grows by adding workers and one failed worker does not stop a camera.
- **92% recognition accuracy over 1,000+ identities** removed 90% of a manual process, and reported incidents fell 40% across 20+ client deployments.

Python · PyTorch · RetinaFace · InsightFace · FAISS · TensorRT · ONNX Runtime · Kafka · Pulsar · ZeroMQ · Redis · FFmpeg · Docker

**Earlier:** Full-stack Developer, LogiGear Corporation (Jun 2018 – Mar 2019) — an Ethereum wallet and smart-contract application in Node.js and React.

## TECHNICAL SKILLS

**LLM systems:** multi-provider gateways with policy routing, continuous health probing and failover across OpenAI, Anthropic, Gemini, Bedrock, Azure and OpenRouter; open-weight models served on vLLM; token metering and cost reconciliation; prompt and agent orchestration with LangChain, Model Context Protocol clients and servers, agent memory; evaluation harnesses and regression gates in CI

**Retrieval & vector search:** embeddings, hybrid keyword-and-vector search with rank fusion, cross-encoder re-ranking and fine-tuning, knowledge graphs (Graphiti); FAISS (IVF, GPU indexes), HNSW, Weaviate, Qdrant, FalkorDB

**ML engineering & serving:** PyTorch, YOLO, RetinaFace and InsightFace, BERT fine-tuning, scikit-learn; TensorRT, Triton, ONNX Runtime, TorchScript, OpenVINO and CoreML with dynamic batching and half precision; FFmpeg video pipelines; edge deployment on K3s

**Languages & backend:** Python (typed, async), Rust, TypeScript / Node.js, Java, SQL; FastAPI, SQLAlchemy 2, Pydantic, pytest, Tokio, Next.js / React; hexagonal architecture, CQRS, event sourcing, multi-tenancy, outbox and idempotency, PCI boundaries, OAuth / OIDC

**Data, storage & cloud:** PostgreSQL (Patroni HA, partitioning, schema per tenant), Redis, MongoDB, DynamoDB, DuckDB, Delta Lake; Kafka, PyFlink, PySpark, Airflow, BigQuery, Databricks; Kubernetes, Helm, Docker, Envoy, OpenTelemetry, Prometheus; AWS, Azure, GCP

## EDUCATION, TEACHING & CERTIFICATIONS

**Education:** BSc Computer Science (High-Quality Program), Ton Duc Thang University, 2015–2019; researcher intern, NewAI Lab (2018)

**Teaching & speaking:** Design Patterns instructor, Ton Duc Thang University (2022); data-analysis mentor, MindX (2023); speaker, AI Workshop, FPT University (2025)

**Certifications:** Google Cloud Technical Series — AI Agents (2025); AWS Machine Learning Foundations (2022)

**Languages:** English (advanced) — the working language of my code, documentation and client work; Vietnamese (native)

## PROJECTS & OPEN SOURCE – REFERENCES

---

Everything on this page is public unless marked otherwise, and each figure comes from the benchmark published in its repository. Pilot Space ([github.com/pilot-space](https://github.com/pilot-space)) is my own open-source line of AI infrastructure, built since January 2026.

**Moon** • [github.com/pilot-space/moon](https://github.com/pilot-space/moon) • [docs pilot-space.github.io/moon](https://docs.pilot-space.github.io/moon) • [moondb](https://crates.io/crates/moondb) on crates.io and PyPI

A Redis-compatible in-memory data store written from scratch in Rust, so existing Redis clients connect unchanged: RESP2 and RESP3, 250+ commands. Vector search (HNSW), BM25 full-text search with hybrid rank fusion, a property-graph engine and transactions that hold across all of them ship in one binary. Against Redis 7.0 it measures 1.62× the throughput on GET and 1.32× on SET with 8–22% less memory per key, and the command families where it loses are published beside the wins. Thirty releases to v0.8.9; single-node production-grade, clustering in alpha.

**Lunaris** • [github.com/pilot-space/lunaris](https://github.com/pilot-space/lunaris) • [docs pilot-space.github.io/lunaris](https://docs.pilot-space.github.io/lunaris) • [lunaris-memory](https://crates.io/crates/lunaris-memory) on crates.io, [@pilot-space/lunaris](https://npmjs.org/package/@pilot-space/lunaris) on npm

An agent-memory engine built on Moon: ingestion, extraction, embeddings, hybrid search and re-ranking behind one API, with the embedding and re-ranking models running in-process so recall makes no network call. Recall across 100,000 documents per scope returns in 19–22 ms at the median and under 25 ms at P99 on a single shard; beyond 100,000 documents it is published as unvalidated. A CI gate scores evidence recall on LongMemEval-S on every retrieval change and fails the build when recall drops. I wrote that gate after finding its predecessor had never once run — all 200 recorded executions were startup failures.

**laya-codex** • [github.com/pilot-space/laya-codex](https://github.com/pilot-space/laya-codex) • [model.huggingface.co/tindang/laya-code](https://model.huggingface.co/tindang/laya-code)

Ranked code retrieval for coding agents. The repository is split into chunks with tree-sitter (14 languages), keyword search finds candidates, and a cross-encoder re-ranker I fine-tuned on git history — where a commit's changed code is the answer to its message — orders them and places the code in the agent's first turn through hooks and the Model Context Protocol. On 60 paired tasks across 3 repositories the right files appeared in the first answer 88% of the time, up from 73%; tool calls per session fell from 5.9 to 2.3, session time by 19% and cost by 14.8%. On a second set of 60 new tasks the cost gain was not significant, and the README says so.

**Hydroa** • [github.com/pilot-space/hydroa](https://github.com/pilot-space/hydroa)

A multi-tenant LLM gateway: one OpenAI-compatible endpoint over OpenAI, Anthropic, Gemini, Bedrock, Azure and OpenRouter. Every row is scoped to a tenant, each tenant's provider keys are encrypted at rest, providers sit behind circuit breakers with health-aware fallback, and a cost ledger reconciles what was metered against what the provider billed.

**ADD** • [github.com/pilot-space/ADD](https://github.com/pilot-space/ADD) • [book pilot-space.github.io/ADD](https://book.pilot-space.github.io/ADD) • [@pilot-space/add](https://npmjs.org/package/@pilot-space/add) on npm, [pilot-space-add](https://pypi.org/project/pilot-space-add/) on PyPI

The spec-first method behind the projects above: the contract is frozen before any code exists, the failing test is committed before the fix, and a build is accepted on a recorded test run rather than a diff that reads well. On 300 real bugs from open-source projects it resolved 226 against 215 for the same agent unassisted, and saw the tests pass before shipping in 274 cases against 53 — at 1.8× the cost. The README states that the gain in bugs fixed is not yet proven.

**yowo** • [github.com/TinDang97/yowo](https://github.com/TinDang97/yowo) • on PyPI

Production YOLO11 and YOLO26 inference and export. It detects the hardware and picks the backend in order — TensorRT, ONNX Runtime on CUDA, CoreML, OpenVINO, CPU — reconnects dropped RTSP streams and tracks objects with ByteTrack. Extracted from the transport platform on page one.

**rjson** • [github.com/TinDang97/rjson](https://github.com/TinDang97/rjson) • [pyrjson](https://pypi.org/project/pyrjson/) on PyPI (experimental)

JSON for Python written in Rust against the CPython C API: parsing 1.29× and serialising 1.46× the speed of orjson over ten reference cases, with byte-identical output.

**dpsutil & the search-engine client** • [github.com/TinDang97/DPS\\_Util](https://github.com/TinDang97/DPS_Util) • [github.com/TinDang97/DPS\\_SearchEngine\\_Client](https://github.com/TinDang97/DPS_SearchEngine_Client)

The library and client behind the camera-AI product at Dai Phat (2019–2021): Kafka and Redis wrappers, Blosc array compression, TurboJPEG coding, a pre-allocated vector pool, and the client for the FAISS search service — flat and IVF indexes, half-precision GPU indexes, GPU-to-CPU moves. 44 releases on PyPI.

**Also** • smaller and earlier work

**Helios**, a Rust agent harness with 12 provider backends, a native Model Context Protocol client and server and sandboxed WebAssembly extensions (private, unreleased) • **Pilot Space**, an AI-augmented software-delivery platform ([github.com/pilot-space](https://github.com/pilot-space), MIT) • **fastapi-multi-tenant**, a multi-tenant FastAPI boilerplate ([github.com/TinDang97/fastapi-multi-tenant](https://github.com/TinDang97/fastapi-multi-tenant)) • **Velos**, a GPU traffic microsimulation in Rust ([github.com/pilot-space/velos](https://github.com/pilot-space/velos)) • **face-anti-spoofing**, started at Vietmoney ([github.com/vietmoney/face-anti-spoofing](https://github.com/vietmoney/face-anti-spoofing)) • **face\_recognition**, RetinaFace and InsightFace ([github.com/TinDang97/face\\_recognition](https://github.com/TinDang97/face_recognition)).

## HOW I EVALUATE

---

- **Gate quality in CI.** In Lunaris every change to retrieval is scored for evidence recall, and the build fails when recall drops.
- **Pair the tasks and state the interval.** The laya-codex benchmark runs the same 60 tasks with and without the tool, reports a 95% confidence interval, and repeats on 60 new tasks.
- **Publish the result that goes against you.** ADD is published with its price, Moon lists the command families where it loses to Redis, and Lunaris marks everything beyond 100,000 documents as unvalidated.
- **Reconcile against an outside source.** At ClickAI I checked metered LLM cost against the providers' own bills, which is how a 30% drift surfaced that no internal metric showed.